

FULL STACK DEVELOPMENT

FASE 5 | AULA 09 -

AZURE CONTAINER APPS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	4
O QUE VOCÊ VIU NESTA AULA?	10
REFERÊNCIAS.....	11

EMSE

O QUE VEM POR AÍ?

Boas-vindas, alunos e alunas.

Nesta aula vocês aprenderão sobre Azure Container Apps, como usá-lo da forma correta, suas principais funcionalidades e quais as similaridades entre ele e o AWS Elastic Container Service, além de entender mais sobre containers.

Bons estudos!

EMSE

HANDS ON

Nesta aula, faremos a criação de um APP Serverless com o Azure Container Apps utilizando nossa imagem que está salva no Azure Container Registry, a fim de aproveitar a mesma imagem, subir a mesma aplicação e analisar as diferenças entre as outras opções de aulas anteriores.

Com a aplicação no ar, vamos aprender a analisar o uso de recursos como CPU, memória RAM e logs dos containers em execução.

EMSE

SAIBA MAIS

Neste documento vamos entender melhor as similaridades entre Azure Container Apps e AWS Elastic Container Service, além de aprender mais sobre containers e como eles funcionam. Um dos problemas mais comuns que encontramos ao rodar um projeto local em nossa máquina é a desorganização e o uso excessivo de memória RAM e CPU, além da configuração necessária para rodar o projeto se misturar com nossos programas já instalados no computador.

Se instalarmos algo errado, ou até mesmo não dimensionarmos corretamente os recursos utilizados no momento em que o projeto rodar, isso pode resultar em uma baita “dor de cabeça”, e muitas vezes é necessário até mesmo formatar o computador.

Podemos enxergar diversos problemas e, pensando nisso, os containers foram criados com o intuito de separar um pedacinho do computador e isolá-lo para rodar projetos, testar sistemas operacionais e corrigir problemas sem afetar a máquina principal e sem a necessidade de precisar redimensionar CPU, RAM, disco e rede, como no caso de uma VM (Virtual Machine = Máquina Virtual).

Dessa maneira, um container é um ambiente isolado dentro de um servidor/máquina que divide e separa seus recursos entre o servidor/máquina dos containers.

Imagine que o servidor é um navio cargueiro e os containers são os containers de carga deste navio: o navio tem a base e a estrutura principal e em cada container de carga há diversos objetos, muitas vezes com conteúdo diferente uns dos outros. No cenário em que um dos containers de carga é danificado ou perdido os demais não são afetados, exatamente por estarem isolados carregando seus próprios objetos.



Figura 1 - Containers
Fonte: Google Imagens (2024)

Sendo assim, um container é totalmente responsável pelo que estiver dentro dele; nesse caso uma aplicação.

Um container pode estar rodando uma aplicação NodeJS com node, npm e assim por diante e outro container rodando uma aplicação em .NET com msbuild. Tudo que estiver instalado no container NodeJS não será compartilhado com o .NET. Diferentemente de uma VM que separa, mas não totalmente, alguns problemas podem ser compartilhados entre as VMs, o que não acontece com os containers.

Para a execução de containers podemos utilizar o Docker, que é uma tecnologia que utiliza o Kernel do Linux e seus recursos, como por exemplo Cgroups e Namespaces, que são empregados para isolar os processos de cada container.

- **Kernel:** é o principal componente do Linux; é responsável pela comunicação entre o hardware e os processos.
- **Cgroup:** é o responsável por colocar limites no uso dos recursos Linux, como CPU, RAM, rede, etc.
- **Namespaces:** são os responsáveis por isolar processos, como rede e grupos de usuários. É o elemento que garante que um host (container) não veja os processos de outro host.

O isolamento é feito devido aos recursos citados.

Agora vamos falar sobre um serviço similar ao Azure Container Apps, só que da Cloud AWS, chamado de “Elastic Container Service” ou “ECS”. Ambos os serviços são serverless (não precisamos nos preocupar com a infraestrutura por trás) e muitas pessoas conhecem o ECS Serverless como “Fargate”. As similaridades são grandes, mas também há algumas diferenças.

Com o AWS ECS, por exemplo, dividimos as seções em três pontos:

1. Cluster.
2. Service.
3. Task Definition.

A primeira camada é o cluster, que é responsável por abrigar os serviços e tasks. Com ele conseguimos concentrar e organizar serviços de determinados grupos; por exemplo, podemos criar um cluster só para aplicações Python, outro só para aplicações .NET e um último para aplicações node e, dentro de cada um, ter somente serviços (containers) relacionados à linguagem em questão.

A segunda camada é o service, que é responsável por concentrar todos os containers relacionados a uma aplicação. Vamos imaginar que um serviço chamado fiap-dotnet-api tenha dez containers rodando; é entrando no serviço fiap-dotnet-api que podemos ver somente os dez containers desse serviço.

E, por último, temos a camada task definition. É nela em que cadastramos tudo relacionado à aplicação, como por exemplo quanta CPU cada container terá, quanta memória, em quais portas a aplicação rodará, configurações de log e configurações de volumes e rede, além de apontar a imagem que a aplicação usará.

Basicamente essa camada é a camada de configuração/inventário da aplicação, é o arquivo de configuração com informações vitais para que o container suba com a imagem correta, portas e afins.

Podemos notar certa similaridade com o Azure e isso é proposital, já que a ideia não é inventar a roda, mas copiar fazendo melhor. Na figura 2 temos um reflexo sobre o que foi descrito.

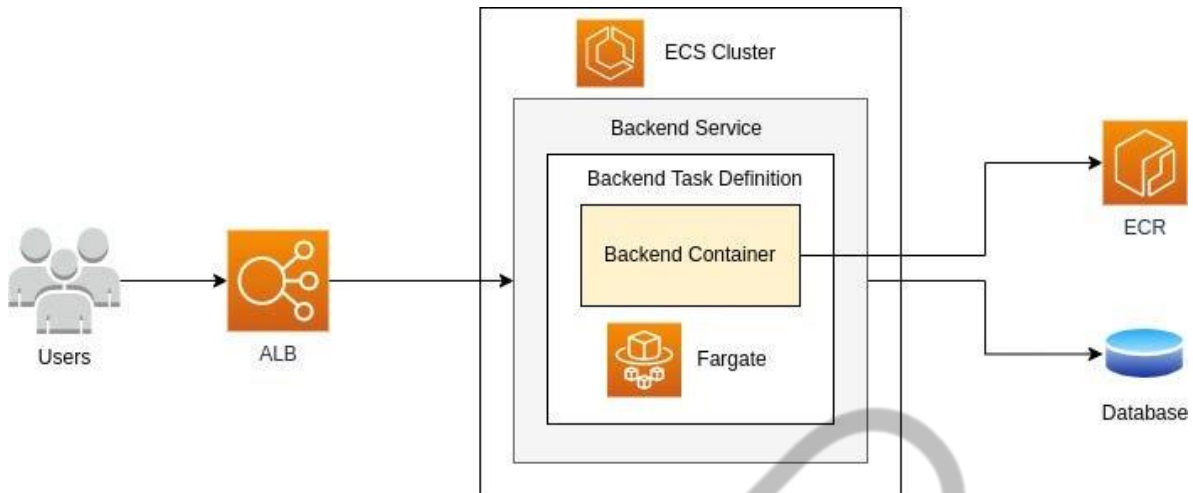


Figura 2 – AWS Fargate
Fonte: Shakya (2022)

Na imagem podemos ver dois serviços que não citamos, sendo eles ALB e ECR.

O ALB, ou Application Load Balance, é o balanceador de carga que fica na frente dos containers e é responsável por dividir o tráfego entre os containers existentes. Vamos imaginar que temos três containers da mesma aplicação rodando e três pessoas acessando a aplicação neste momento.

O ALB faz o trabalho de balancear a carga de cada container, direcionando cada um dos três usuários para cada container; a conta não é exata, mas basicamente assim funciona e trabalha um Load Balance.

O outro serviço vocês já conhecem do Azure; lá ele se chama Azure Container Registry, isso mesmo, o ECR ou Elastic Container Registry. É o repositório de imagens que serão usadas para subir e rodar os containers.

Ele é mais um repositório disponível dentre outros para hospedarmos nossas imagens e não fica preso à AWS. Podemos usar o repositório da AWS para empregarmos em um deploy do Azure Container Apps; ele precisa estar autenticado, mas é possível atualmente.

Também temos a opção de rodar um container no ECS usando uma EC2 (Elastic Compute Cloud), em outras palavras uma VM. Outra similaridade com o nosso serviço Azure Container Instance.

Há diversas comparações entre serviços de ambas as clouds; então, se você aprende no Azure o conceito por trás e como funciona se aplica na AWS, Google e outras clouds, a única diferença principal está na disposição entre os serviços e layout de cada cloud, mas no geral o envio da imagem para ECR e para ACR é basicamente da mesma maneira, assim como mandar o deploy via pipeline para AWS e para Azure são similares também, o que muda são os comandos, mas os passos são bem similares.

Então, experimentem outras clouds, ambas possuem o Free Tier anual para testar os principais serviços de forma gratuita utilizando a menor arquitetura de cada cloud.

Espero que tenham aproveitado essa aula e aprendido mais sobre cloud.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você viu como funcionam os containers e quais as similaridades entre Azure Container Apps e AWS Elastic Container Service (Fargate).

EMAN

REFERÊNCIAS

AWS. **O que é o Amazon Elastic Container Service?**. 2024. Disponível em: https://docs.aws.amazon.com/pt_br/AmazonECS/latest/developerguide/Welcome.html. Acesso em: 29 abr. 2024.

MICROSOFT BUILD. **Visão geral sobre os Aplicativos de Contêiner do Azure**. 2023. Disponível em: <https://learn.microsoft.com/pt-br/azure/container-apps/overview>. Acesso em: 29 abr. 2024.

SHAKYA, S. **Setup Application Load Balancer and Point to ECS — Deploy to AWS ECS Fargate with Load Balancer (Last Part)**. 2022. Disponível em: <https://sakyasumedh.medium.com/setup-application-load-balancer-and-point-to-ecs-deploy-to-aws-ecs-fargate-with-load-balancer-4b5f6785e8f>. Acesso em: 29 abr. 2024.

TUTORIALS DOJO. **AWS vs Azure Services Comparison Cheat Sheet**. 2023. Disponível em: <https://tutorialsdjo.com/aws-vs-azure-services-comparison>. Acesso em: 29 abr. 2024.

PALAVRAS-CHAVE

Palavras-chave: Container. Azure Container Apps. AWS. ECS. Fargate.

EMSE

POSTECH